

VEILLE TECHNOLOGIQUE FRONT

1. Contexte du projet

Le projet consiste à développer une application de concentration **desktop et mobile synchronisée**, axée sur la **productivité gamifiée**. L'utilisateur gagne des points en réalisant des sessions de focus, qu'il peut ensuite utiliser pour personnaliser un **monde 3D interactif** et débloquer divers avantages.

L'application intègre plusieurs dimensions : - gestion du focus et du temps de travail, - blocage de distractions (applications, notifications, réseaux sociaux), - analyse de l'activité utilisateur, fonctionnalités sociales et collaboratives, - univers graphique 3D immersif.

Dans ce contexte, le choix des technologies frontend est stratégique afin de garantir de bonnes performances, une expérience utilisateur qualitative et une maintenabilité sur le long terme

2. Objectifs de la veille technologique

Cette veille technologique a pour objectif de :

- Evaluer la faisabilité des fonctionnalités via différentes technologies frontend
- Comparer les solutions existantes pour une application desktop moderne
- Identifier une architecture frontend adaptée aux contraintes du projet
- Proposer une stack technologique cohérente, réaliste et maintenable

Analyse des technologies desktop

a. Electron

Electron est un Framework permettant de développer des application desktop multiplateformes à partir des technologies web (html, CSS, JS)

Avantages	Inconvénients
- Large écosystème et forte communauté - Intégration facile avec React, Vue ou Svelte - Support natif des notifications système - Compatibilité avec Three.js pour le rendu 3D - Rapidité de développement et partage de code avec le mobile/web	- Accès limité aux fonctionnalités systèmes avancées - Impossibilité native de bloquer des applications système - Accès restreint aux évènements clavier/souris globaux - Performances dépendantes d'une bonne optimisation

Conclusion : Electron seul ne suffit pas pour couvrir l'ensemble des besoins du projet, notamment pour les interactions systèmes avancées.

b. Electron avec modules natifs

Une architecture hybride peut être mise en place, combinant Electron avec des modules natifs spécifiques au système d'exploitation

Principes	Avantages	Inconvénient
- Electron gère l'interface utilisateur - Des modules natifs (C++, Rust, Swift, ...) gèrent les interactions système - Communication via IPC (Inter-	- Accès aux API Système (blocage d'applications, monitoring, permissions) - Conservation d'une UI moderne et multiplateforme - Flexibilité et évolutivité	- Complexité technique accrue - Maintenance spécifique par OS

Process Communication)		
---------------------------	--	--

Conclusion : Cette approche représente le meilleur compromis entre puissance fonctionnelle et expérience utilisateur.

Alternatives étudiées

Technologie	Analyse
Tauri	Léger et performant, mais APIs systèmes encore limitées pour des besoins avancés
Flutter Desktop	UI performante, mais intégration 3D web et OS plus complexe
Qt	Très puissant, mais lourd et couteux en temps d'apprentissage
.NET MAUI	Peu adapté à la 3D et à la gamification

Electron reste toujours la solution la plus adaptée au contexte du projet

3. Choix des bibliothèques graphiques

a. Bibliothèques UI

Créer une bibliothèque graphique personnalisée complète représente un risque élevé en termes de temps, d'accessibilité et de maintenance.

Solution retenue :

- Approche hybride
- Utilisation d'une bibliothèque UI headless
- Personnalisation avancée via la direction artistique et de maintenance

Bibliothèques recommandées :

- **Radix UI** (accessibilité et composants non stylés)
- Ou **MUI Base**

Rendu 3D et animations :

- **Three.js** pour le rendu du monde 3D
- **React Three Fiber** pour une interaction naturelle avec **React**
- **Framer Motion** pour les animations d'interface

Cette combinaison permet un rendu immersif tout en conservant une bonne productivité

4. Architecture frontend

a. Organisation du projet

L'architecture modulaire par domaines fonctionnels est recommandée :

- Séparation des fonctionnalités métiers
- Mutualisation des composants et services
- Meilleure lisibilité du code

```
src/  
├─ app/           (bootstrap, routing)  
├─ features/      (focus, shop, world, social)  
├─ shared/        (UI, hooks, utils)  
├─ services/      (API, WebSocket, IPC)  
├─ stores/        (state global)  
└─ assets/
```

b. Avantages

Cette organisation permet :

- Une meilleure collaboration dans l'équipe
- Une évolutivité du projet
- Une maintenance facilitée

5. Performance et optimisation

Pour garantir une application fluide il faudrait :

- Un chargement différé des fonctionnalités lourdes (monde 3D)
- Optimisation des assets graphiques
- Limitation du rendu 3D hors interaction

- Une utilisation des Web Workers pour des calculs intensifs

Electron peut offrir de bonnes performances si ces bonnes pratiques sont respectées

6. Gestion de l'état et communication

a. State management

- **Zustand** pour la gestion de l'état global,
- Séparation claire des états UI, utilisateurs, sessions de focus et de monde 3D

b. Communication frontend / backend

- Utilisation des **API REST** pour une communication classique
- **Web Sockets** pour les sessions synchronisées et le temps réel
- **IPC Electron** pour les interactions système

7. UX, accessibilité et direction artistique

a. Expérience utilisateur

- Interface claire et non culpabilisante
- Feedback visuel immédiat
- Animations légères et optionnelles

b. Accessibilité

- Navigation clavier complète
- Contrastes conformes aux normes WCAG
- Gestion du focus et des tailles de texte

c. Cohérence visuelle

- Utilisation de design Tokens
- Thèmes clairs et sombre
- Séparation visuelle entre UI fonctionnelle et Univers 3D

8. Maintenabilité et pérennité

Le stack proposé repose sur des technologies largement adoptées, bien documentées et disposant d'une communauté active. Elle facilite la maintenance, les mises à jour et l'évolution de l'interface dans le temps.

9. Conclusion et recommandation

Au regard des contraintes techniques, graphiques, humaines et temporelles du projet, la solution la plus pertinente serait :

Electron + React + Type Script + Three.js, enrichie par des modules natifs pour les fonctionnalités système avancées

Cette stack offre un excellent compromis entre performances, ergonomie, richesse fonctionnelle et maintenabilité, tout en restant réaliste pour un projet académique et professionnel